Assembly Programming → apart from arithmetic, logical ops, branches & loops, concept of memory alignment

Process Management → Life cycle, data structures, threads, scheduling

Structure of OS

Bootup Process

Process Management

How can processes or coordinate among each other? → Since processes are isolated, then why is process communication needed?

⟹ Inter-process communication (IPC) is needed to coordinate access to shared resources.

All data communication existing today uses

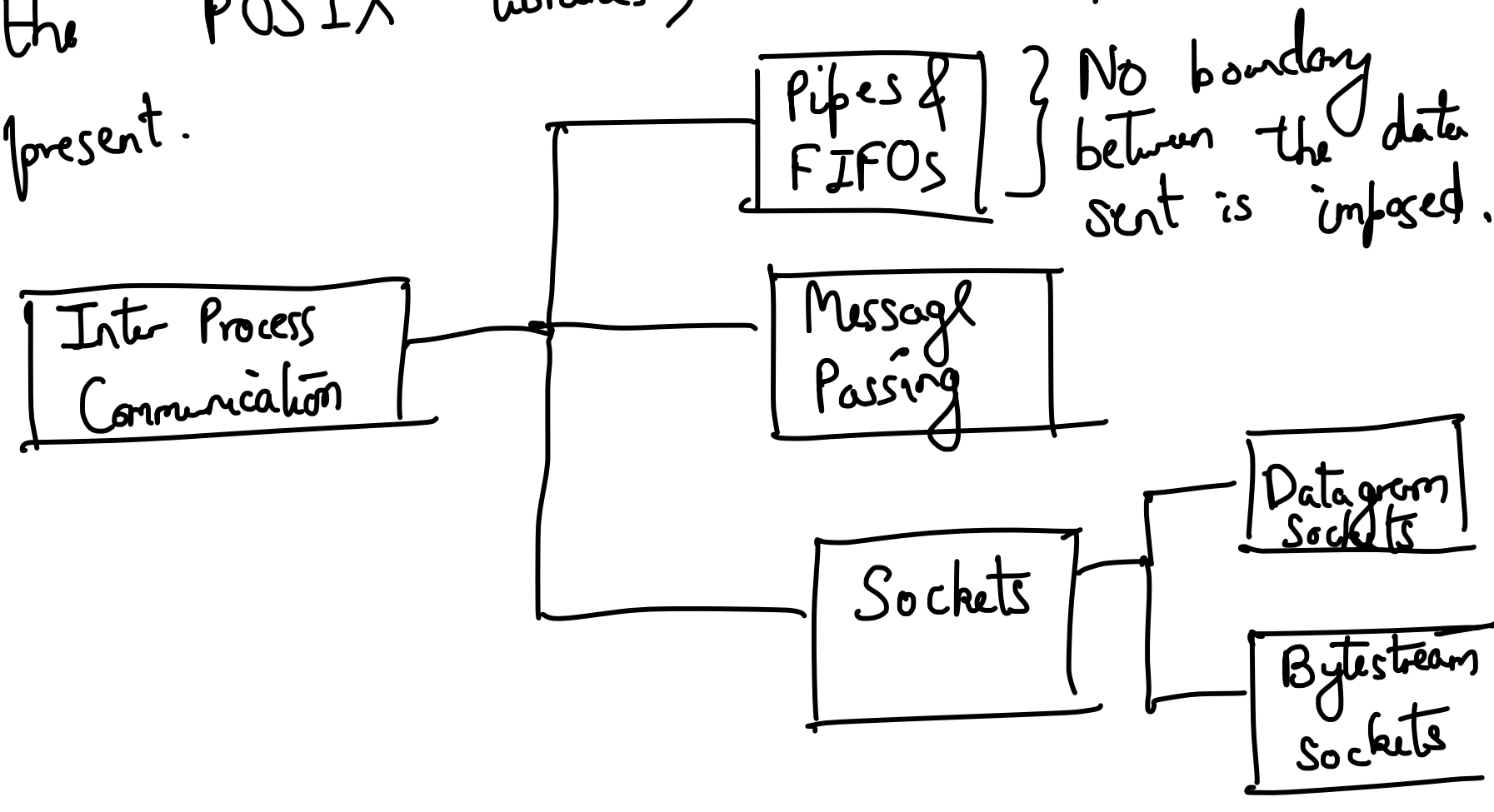
IPC.

POSIX is the common standard used by all UNIX-based OS to provide process communication mechanisms.

System V standard coming from AT&T's version of UNIX.

$\Rightarrow$ Also provided mechanisms of IPC.

Most UNIX systems have support for IPC from two different libraries. We cover only the POSIX libraries, wherever duplication is present.



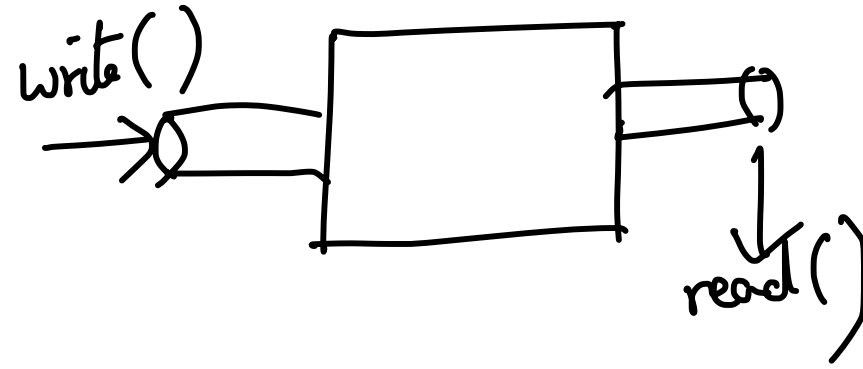
Communication through ① files is considered to degrade performance. Most of the IPC mechanisms in UNIX use the standard file-handling mechanisms The advantage of using it is that there is no need to build familiarity with a new set of system calls.

② Shared Memory: → If you need to use many read's and writes, then the file-handling mechanisms can reduce performance due to context-switches. If you use shared memory, then reading and writing is similar to reading and writing from an array. But Consistency has to be ensured by the user processes through use of proper locks.

Pipe ("Anonymous" Pipe)

→ Virtual or pseudo-file which can be used for IPC.

```
int fd[2];
fd= pipe();
write(fd[0]);
read(fd[1]);
```




```
pid = fork();
if (pid == 0) {          /* Child Process */
    char *buf = "abc";
    write(fd[0], buf);
    :
}
else {                   /* Parent Process */
    char *buf;
    buf = malloc(100);
    read(fd[1], buf);
```

}

(1) The disadvantage of this pipe is - that there is no way of communicating across processes that are not related.

(2) Each process can only read or write but not both. Communication using a single pipe is unidirectional. Bidirectional communication would require two different pipes.

FIFO or Named pipe can be used to communicate across processes that are not related.

```
fifo("abc");
```

Both the processes would need to use the same name so that the kernel can identify that they want to communicate

with each other. After opening using the system call, read/write/close can be used as usual. Other facilities are similar to the original pipes.

Pipes do not guarantee that they would read/write the entire data at a time.

ret_val = write (fd, buf, buf_len)
512/1024, 4096

/* ret_val indicates how many bytes have been written */

```
tot_char = 0;
while (tot_char < 512) {
    ret_val = write (fd, buf[tot_char], 512 - tot_char);
    tot_char += ret_val;
}
close (fd);
```

Read to a pipe is blocking in nature.

```
while (tot_char < 1024) {
    ret_val = read (fd, buf[tot_char], 1024 - tot_char);
    tot_char += ret_val;
}
```

read() will silently wait for the next characters to be written to the pipe.

Exercise: → Send one line of text from one process to another using pipe and FIFO